



POLITECNICO
MILANO 1863

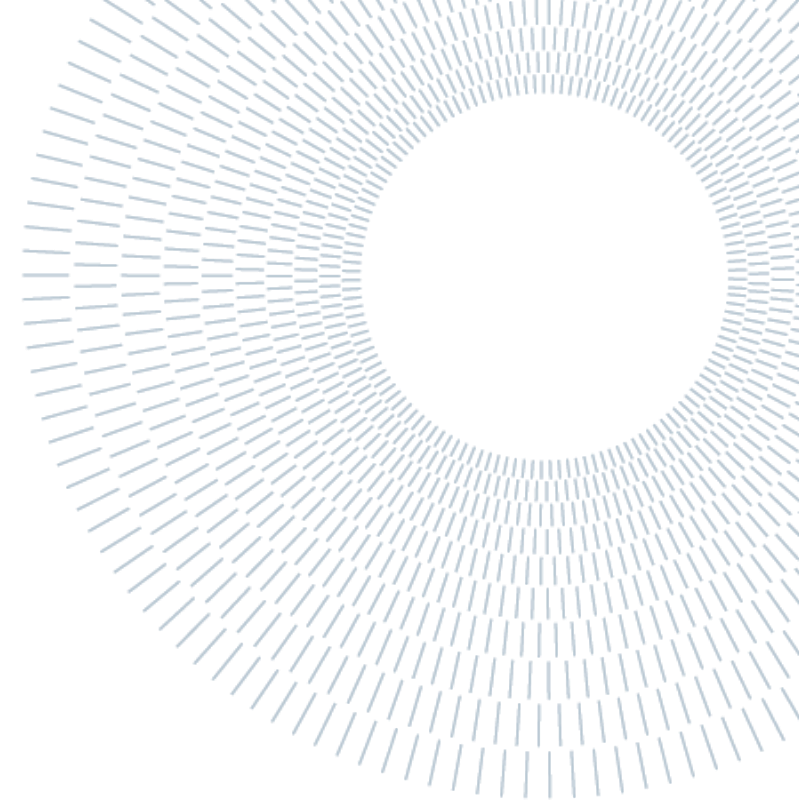
SPACECRAFT ATTITUDE DYNAMICS AND CONTROL

Lab 2 – Numerical Integration in Simulink

PIETRO BRUSCHI, MASSIMILIAMO BUSSOLINO, ANNACHIARA IPPOLITO, ANDREA COLAGROSSI

A.Y. 2025-2026

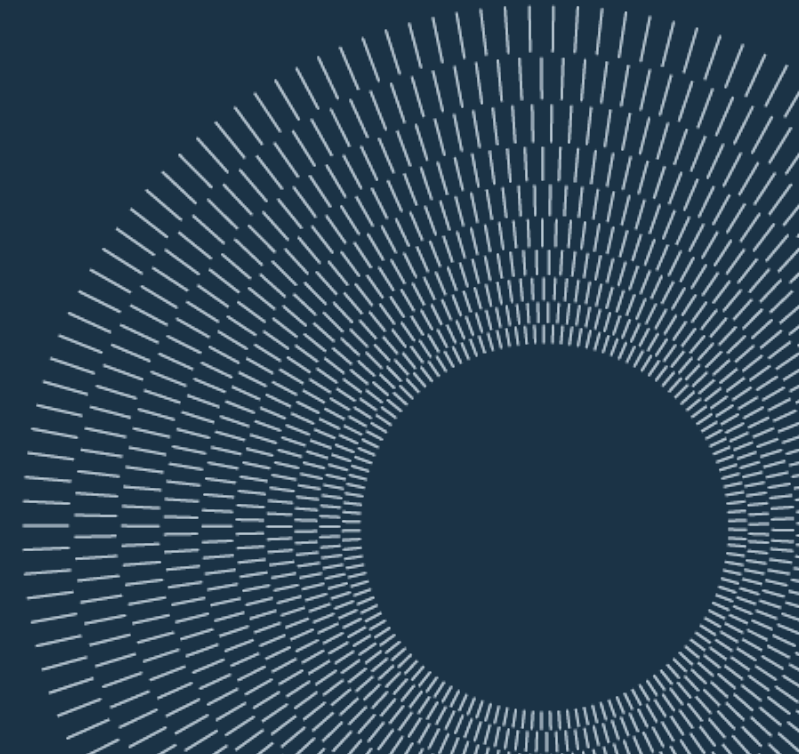
- Common Blocks Settings
- Numerical Integration in Simulink



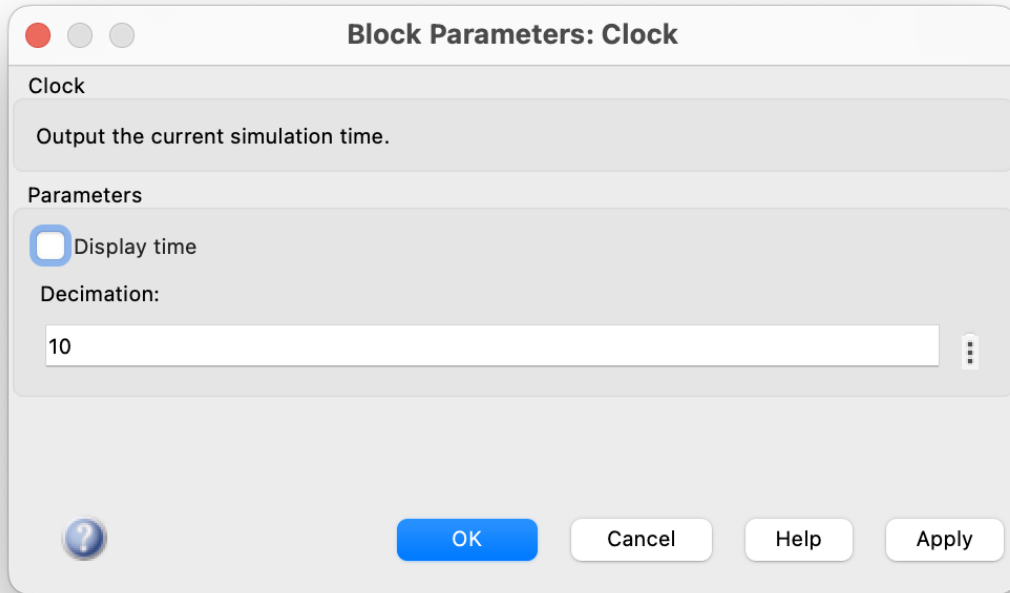
Common Blocks Settings



POLITECNICO
MILANO 1863



Common Blocks Settings | Clock

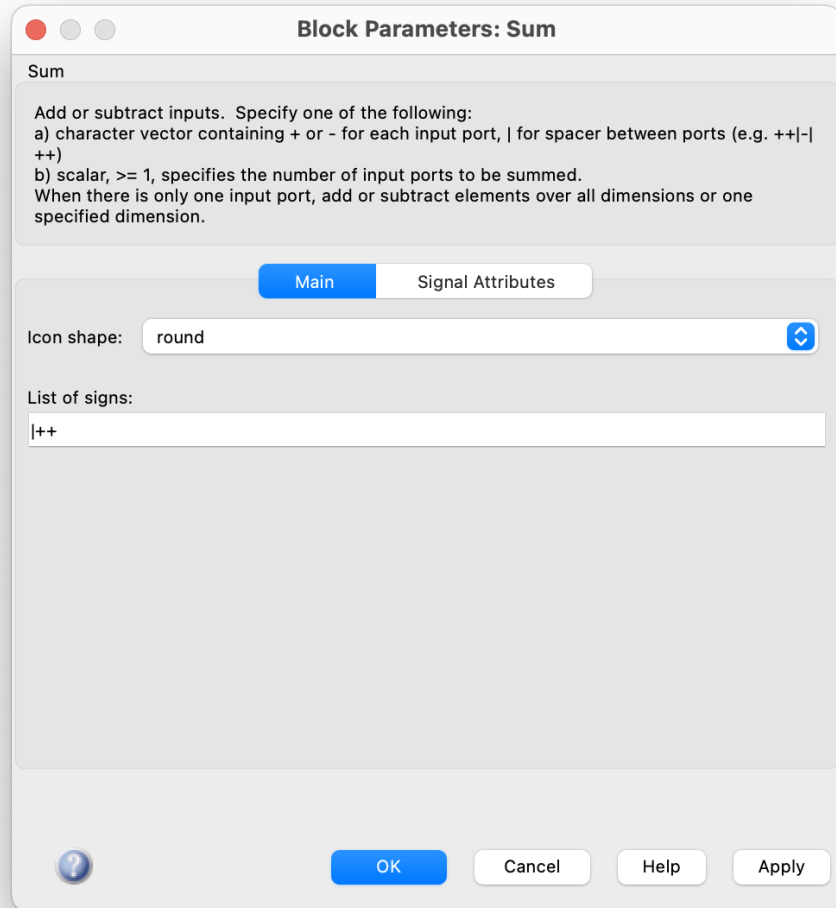


The clock block has virtually no settings, as it only returns the **current simulation time**.

Note: simulation time is **NOT** the actual time elapsed (in real world). Simulations in Simulink are usually faster or slower than the real world clock.

The only available option is to display the simulation time in the block icon. The decimation parameter defines how often the icon should update.

Common Blocks Settings | Sum



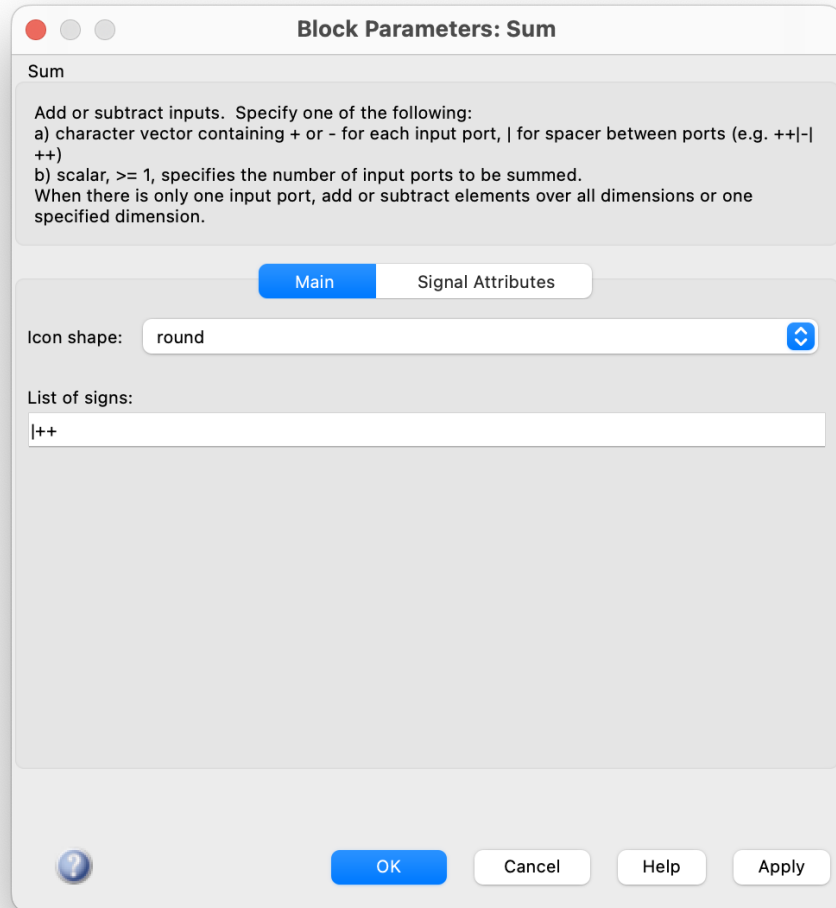
The Sum block – *as the name suggests...* – performs the signed sum operation of its inputs.

Double-clicking on the sum block shows its parameters.

The **Main** tab contains the following settings:

- **Icon shape:** round or rectangular. Rectangular is suggested for many sums
- **List of signs:** the order of signs to be considered for the corresponding inputs.
Can be + (positive), - (negative), or | (add a placeholder to space the ports out)

Common Blocks Settings | Sum

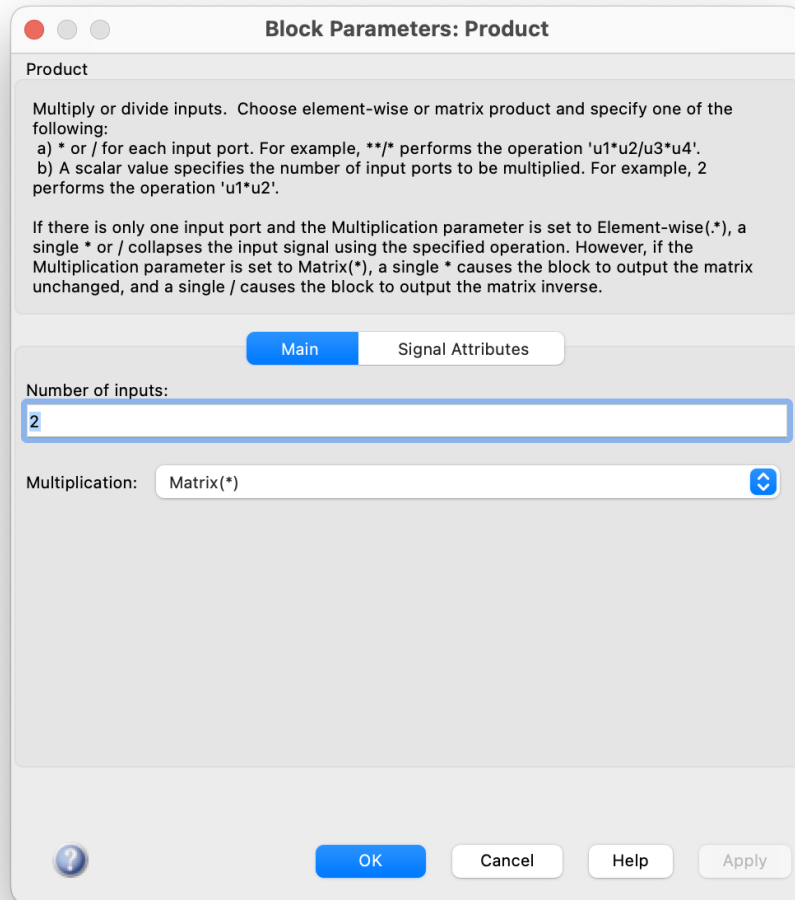


If a single input sign is specified, the sum blocks acts as the MATLAB sum function, performing the sum of all elements of a matrix or vector along the specified dimension(s).

That is, for an input vector $\mathbf{v}$, the block returns:

- $v[1] + v[2] + v[3] + \dots + v[\text{end}]$ if sign is +
- $0 - v[1] - v[2] - \dots - v[\text{end}]$ if sign is -

Common Blocks Settings | Product



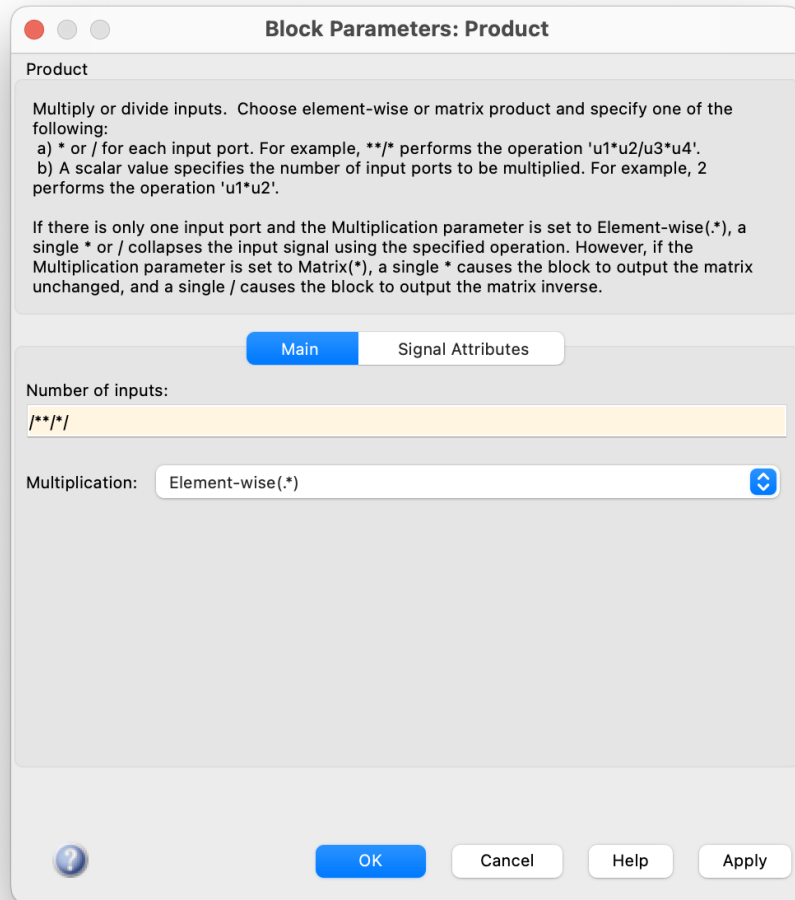
The **product block** can be configured to act as:

- **Element-wise product:** the multiplication is performed element-by-element.
This corresponds to the MATLAB operator `.*`
- **Matrix product:** the multiplication is intended as matrix multiplication (row-by-column).
This corresponds to the MATLAB operator `*`

⚠ Size and dimensions of the inputs must be compatible

Multiple multiplications can be chained by setting a higher number of inputs.

Common Blocks Settings | Product



Element-wise product

Can be used for:

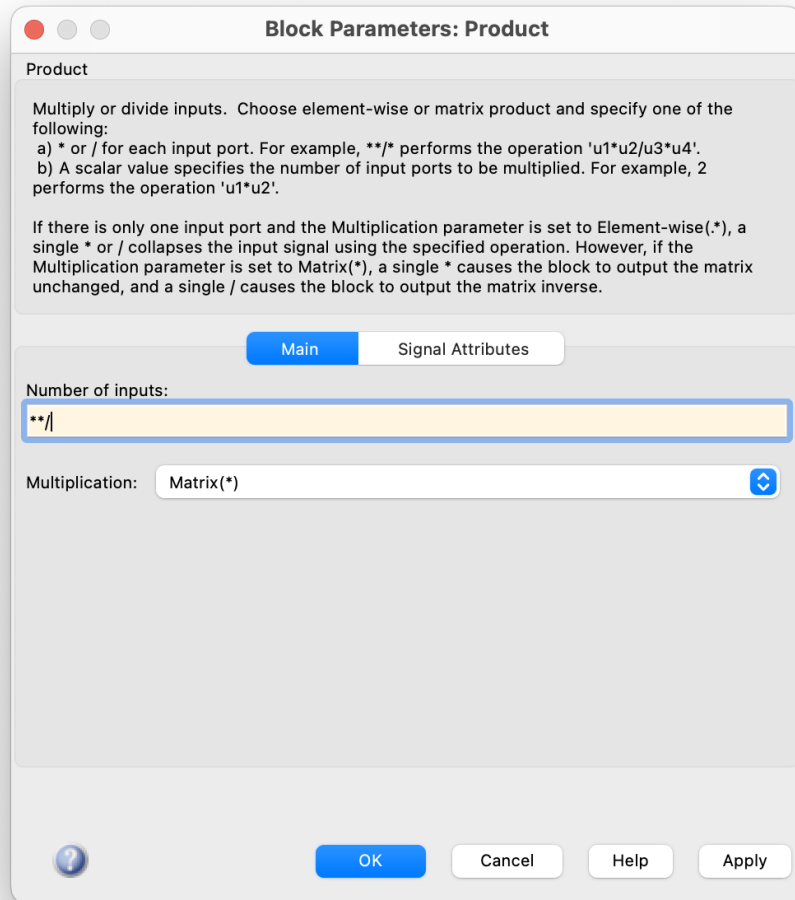
- Scalar x Scalar
- Scalar x Nonscalar
- Nonscalar x Nonscalar only with compatible sizes

The number of inputs can contain:

- **A number:** the number of inputs
- **A string containing * or /:** the length and position determines the number of inputs and the multiply or divide operation.

Es: `/**/*` is 4 inputs in which the first and last are divided

Common Blocks Settings | Product



Matrix product

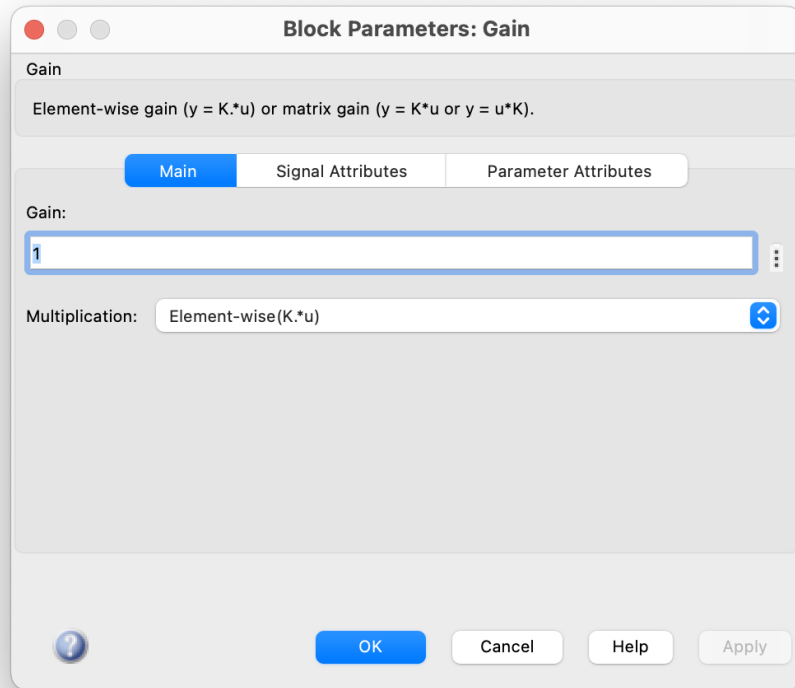
Can be used for the Matrix multiplication. Dimensions should be consistent with the matrix multiplication operation (i.e., $[3 \times 5] \times [5 \times 3]$ is defined $[3 \times 3]$, $[4 \times 6] \times [2 \times 4]$ is not defined).

Again, the number of inputs can contain:

- **A number:** the number of inputs
- **A string containing * or /:** the length and position determines whether the operation involves the original matrix or its inverse.

A single / returns the inverse of the matrix.

Common Blocks Settings | Gain

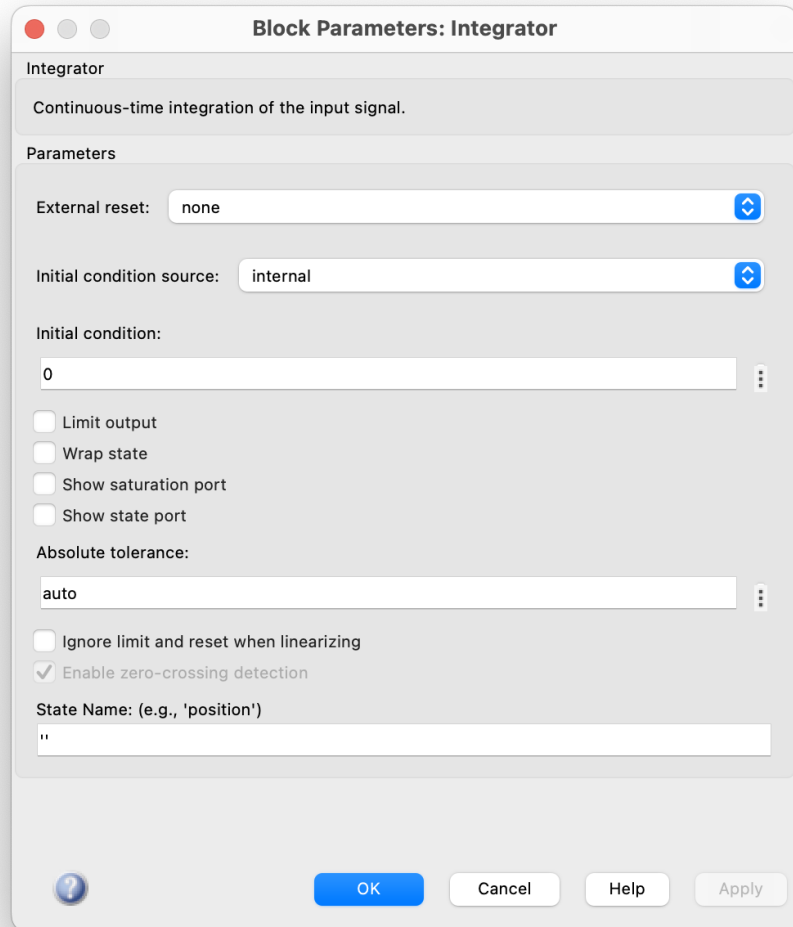


The Gain block is similar to the product block, but one of the inputs is fixed in the Block Parameter interface. It can be both a scalar and nonscalar.

The following operations are supported:

- Element-wise multiplication
- Matrix multiplication $K * u$
- Matrix multiplication $u * K$
- Matrix multiplication $K * u$ with u as a vector – identical to the second, but more restrictive in terms of how dimensions are determined

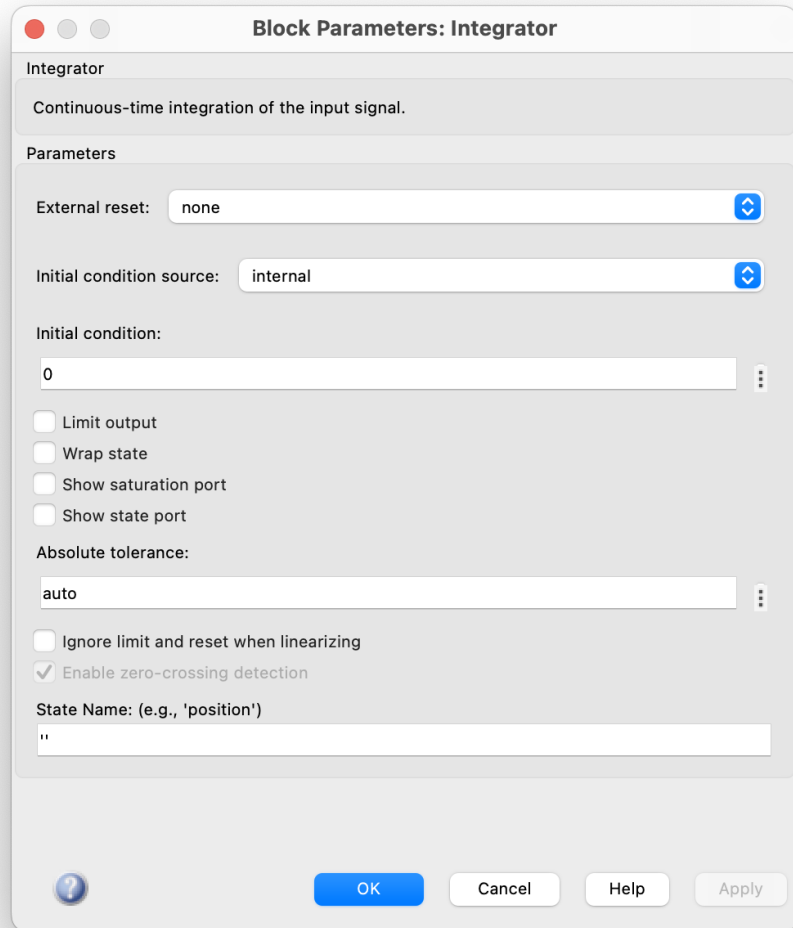
Common Blocks Settings | Integrator



The integrator block is probably the most important block in Simulink. Options:

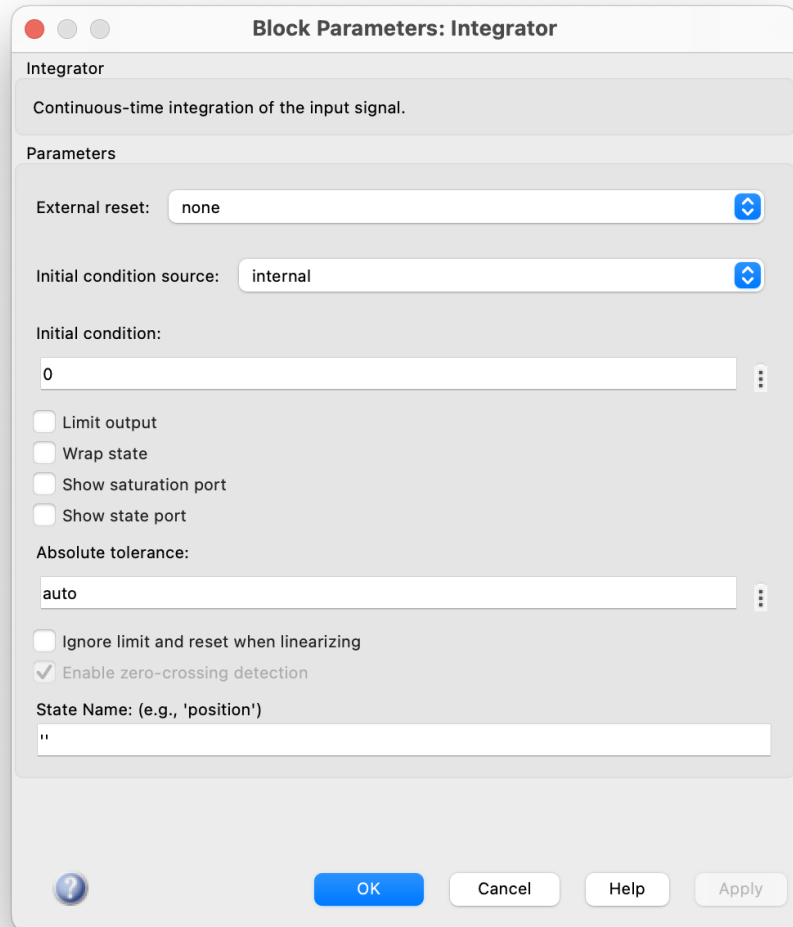
- **External reset:** we can choose to reset the output back to the initial condition according to an external signal.
- **Initial condition source:** internal or external. Internal is «hard-coded» in the block. External can come from any output. External conditions can be useful with reset to change the initial conditions upon resetting.
- **Initial condition:** Appears only if the source is set as internal. The initial condition from which to integrate.

Common Blocks Settings | Integrator



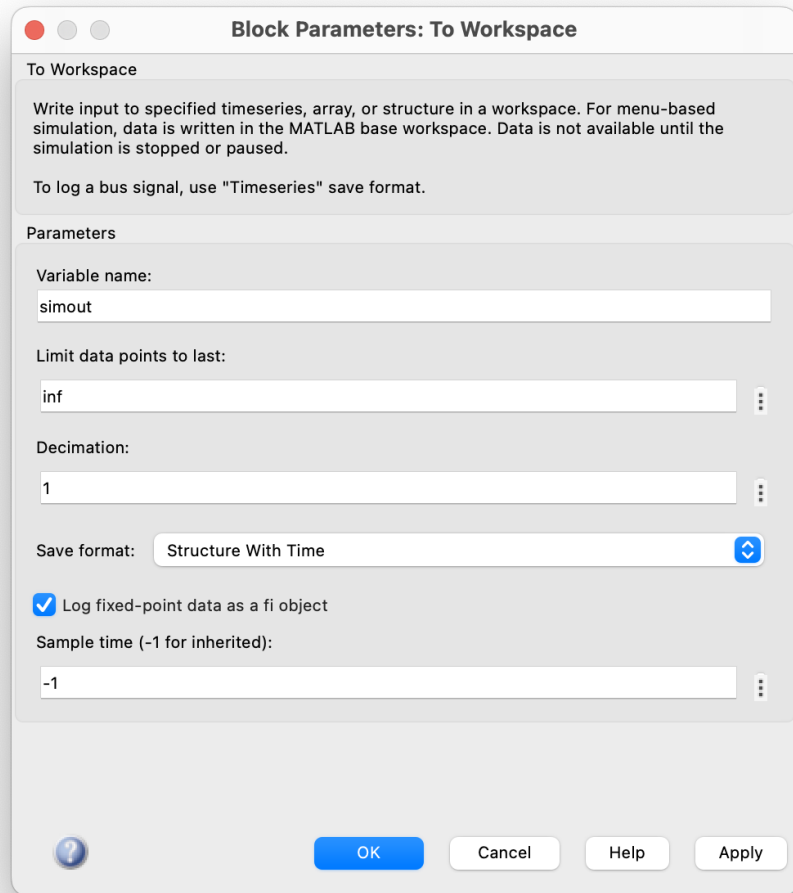
- **Limit output:** we can set boundaries to the output. The output signal will be limited to the specified values.
- **Wrap state:** wraps the output in a specified interval. Useful for angles (i.e., 4π is just 0...)
- **Show saturation port:** If limits are enabled, an additional port that signals whether the output is saturated (limited) or not can be shown.
- **Show state port:** an additional port appears showing the state. The state is equal to the output but is computed before resetting. Useless in practice.

Common Blocks Settings | Integrator



- **Absolute tolerance:** only used if the model is configured with *variable-step solvers*. Allows to specify a different numerical tolerance (precision) for the integration.
- **Ignore limit and reset when linearizing:** this settings is specific when linearizing systems, but useless in practice.
- **Enable zero-crossing detection:** enables the simulation to perform additional, intermediate steps to look for zeros in the output variable. This is disabled by default in fixed-step solvers
- **State name:** an useful label to mark the output signal – use it!

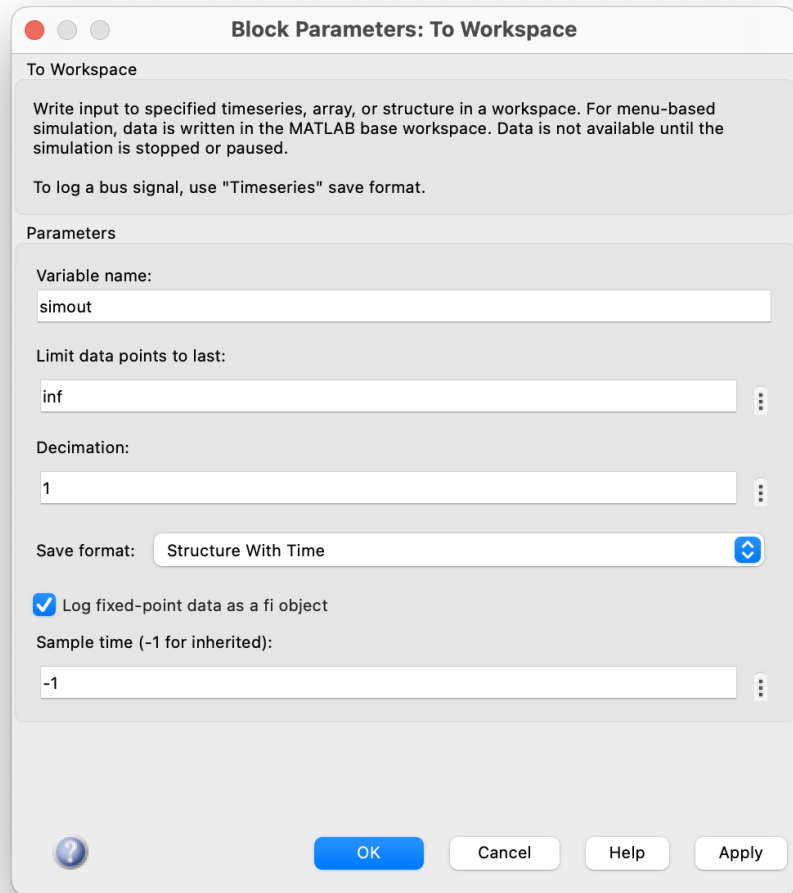
Common Blocks Settings | To Workspace



The *To Workspace* block allows transferring the simulation outputs to MATLAB.

- **Variable name:** the field name of the `out` structure associated to the signal in the MATLAB workspace.
- **Limit data points to last:** we can choose to save the last N data points only.
- **Decimation:** we can choose to save the data at each N timesteps

Common Blocks Settings | To Workspace



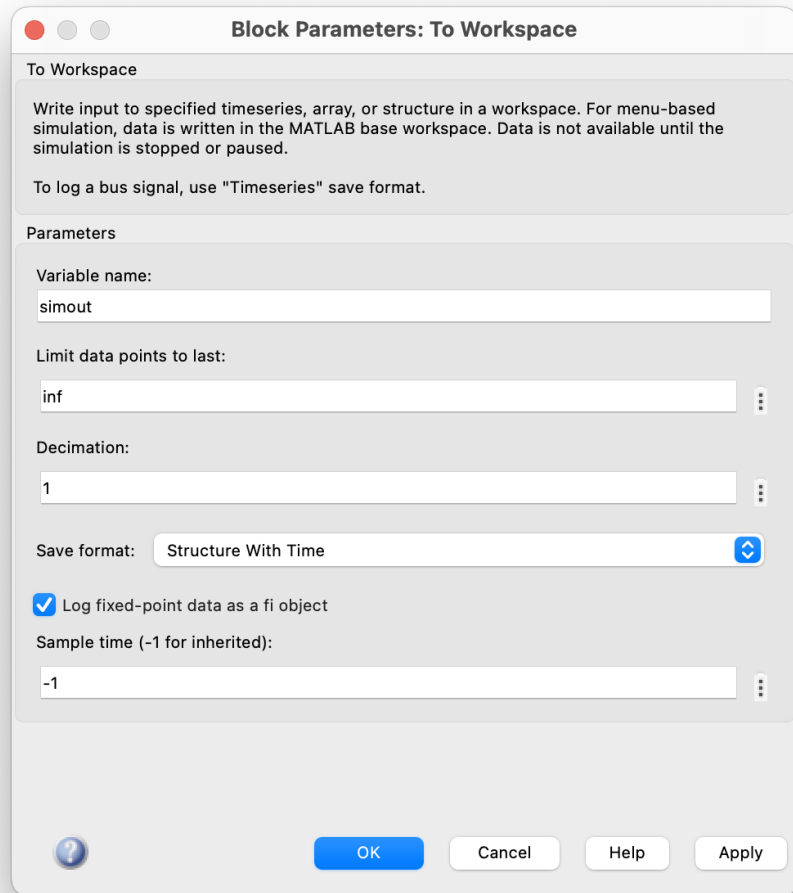
- **Save format:** the signal will appear in the MATLAB workspace as:

- *Structure with time:* a structure with the fields **time** and **signals**. The **signals** sub-field contains the field values, dimensions, and label

```
out
|-- simout
    |-- time
    |-- signals
        |-- values
        |-- dimensions
        |-- label
```

- *Structure:* same as before, but the time array is empty

Common Blocks Settings | To Workspace

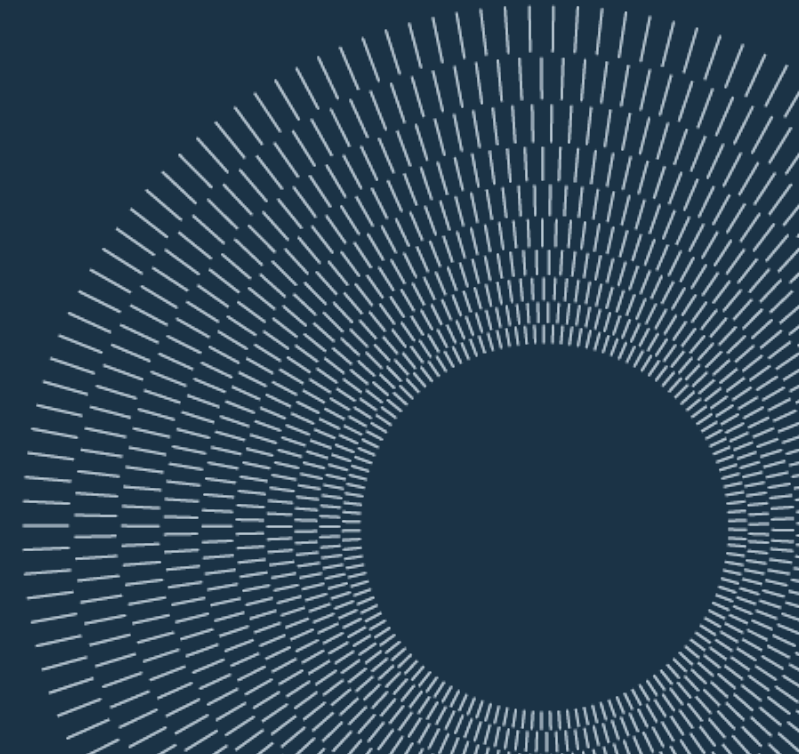


- **Save format:** the signal will appear in the MATLAB workspace as:
 - *Array*: the raw data signal, with no time information
 - *Timeseries*: a complex MATLAB class that contains additional information on the signal
- **Log fixed point data as fi object:** a setting specific for fixed-point arithmetics. MATLAB usually uses floating-point arithmetics.
- **Sample time:** if we want this block to be sampled at a specific sample time. If inherited, it inherits the sample time of the input signal.

Numerical Integration



POLITECNICO
MILANO 1863



Numerical Integration



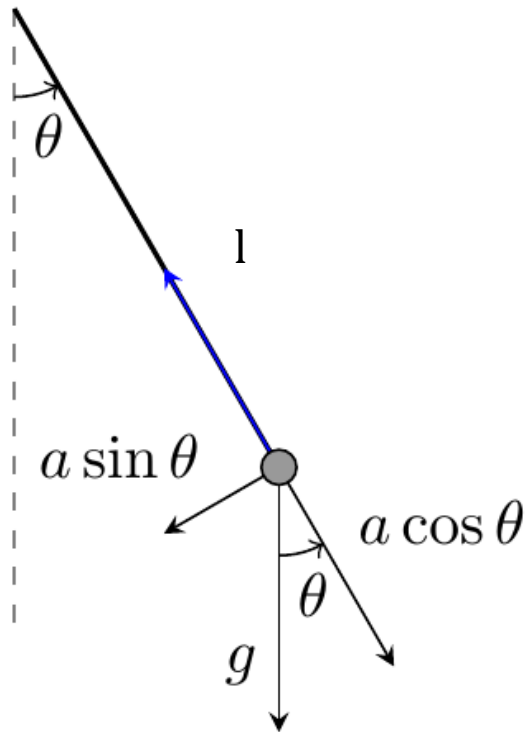
Computers are finite machines.

They do not reason in a continuous fashion: they are digital machines that reason in finite steps.

When using any solver on a computer, we are accepting an approximation: the solution that will be given will be true only up to a certain numerical precision.

Why?

Example: the simple pendulum

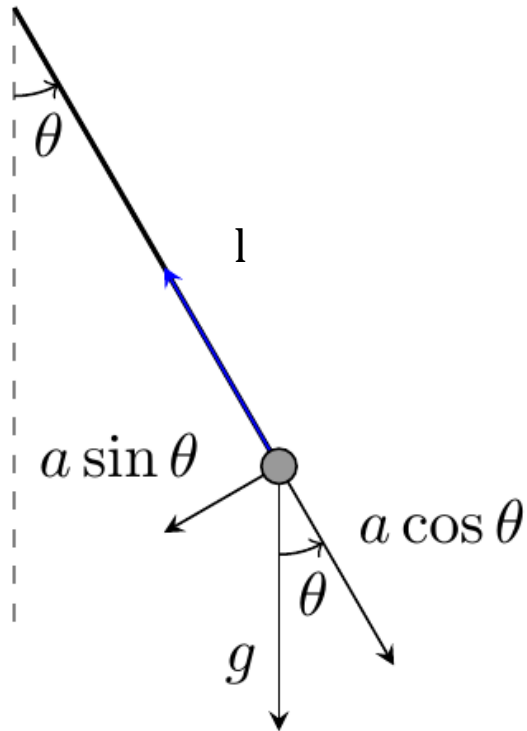


Let's consider a generic pendulum. We know that, without friction, we can model its angular coordinates with the second-order differential equation:

$$\ddot{\theta} = -\frac{g}{l} \sin \theta$$

How can we solve this equation?

Example: the simple pendulum



For small angles, we can approximate the equation as:

$$\ddot{\theta} = -\frac{g}{l} \sin \theta \simeq -\frac{g}{l} \theta$$

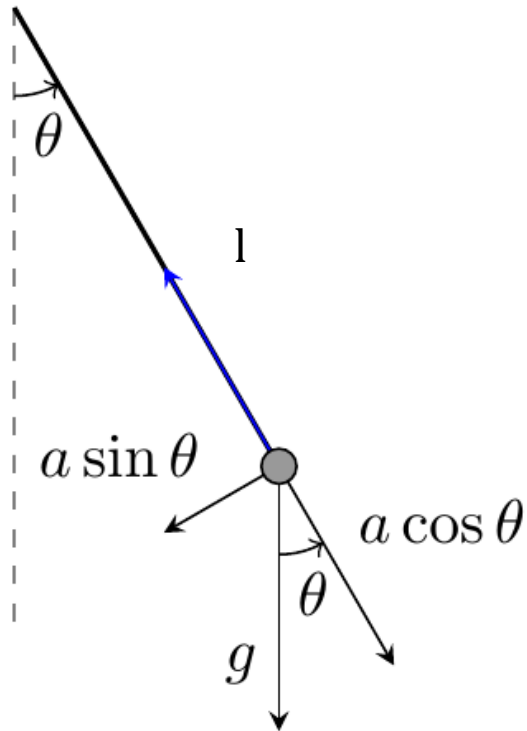
And we know that the solution has this form:

$$\theta(t) = a \sin\left(\sqrt{\frac{g}{l}} t\right) + b \cos\left(\sqrt{\frac{g}{l}} t\right)$$

For a null initial angle and a non-null initial speed ω_0 :

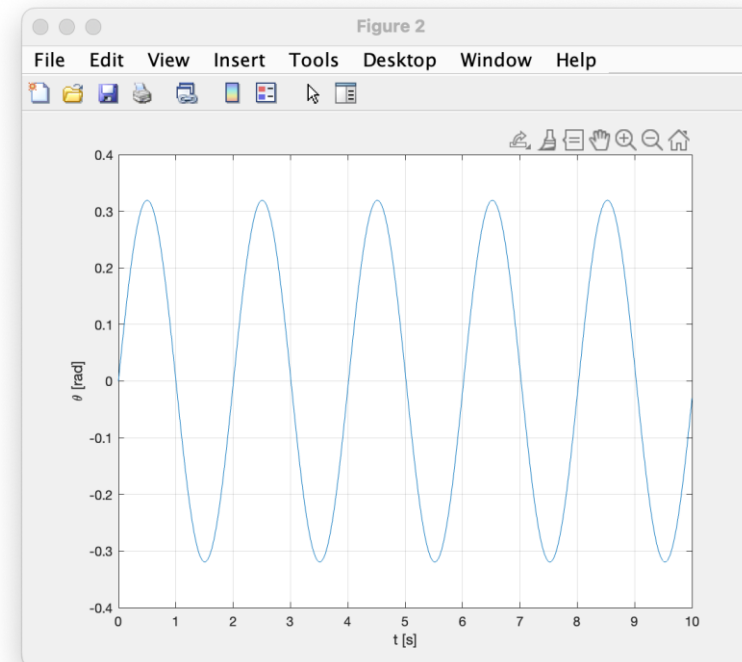
$$\theta(t) = \omega_0 \sqrt{\frac{l}{g}} \sin\left(\sqrt{\frac{g}{l}} t\right)$$

Example: the simple pendulum

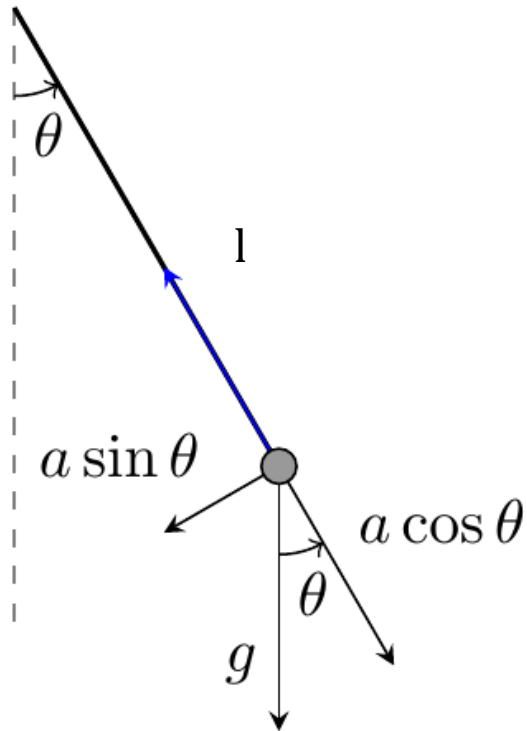


$$\theta(t) = \omega_0 \sqrt{\frac{l}{g}} \sin\left(\sqrt{\frac{g}{l}} t\right)$$

This is an analytical solution. We can know the value of theta just by evaluating this expression, at any time instant.



Example: the simple pendulum



What if we want to solve the non-approximated equation?

$$\ddot{\theta} = -\frac{g}{l} \sin \theta$$

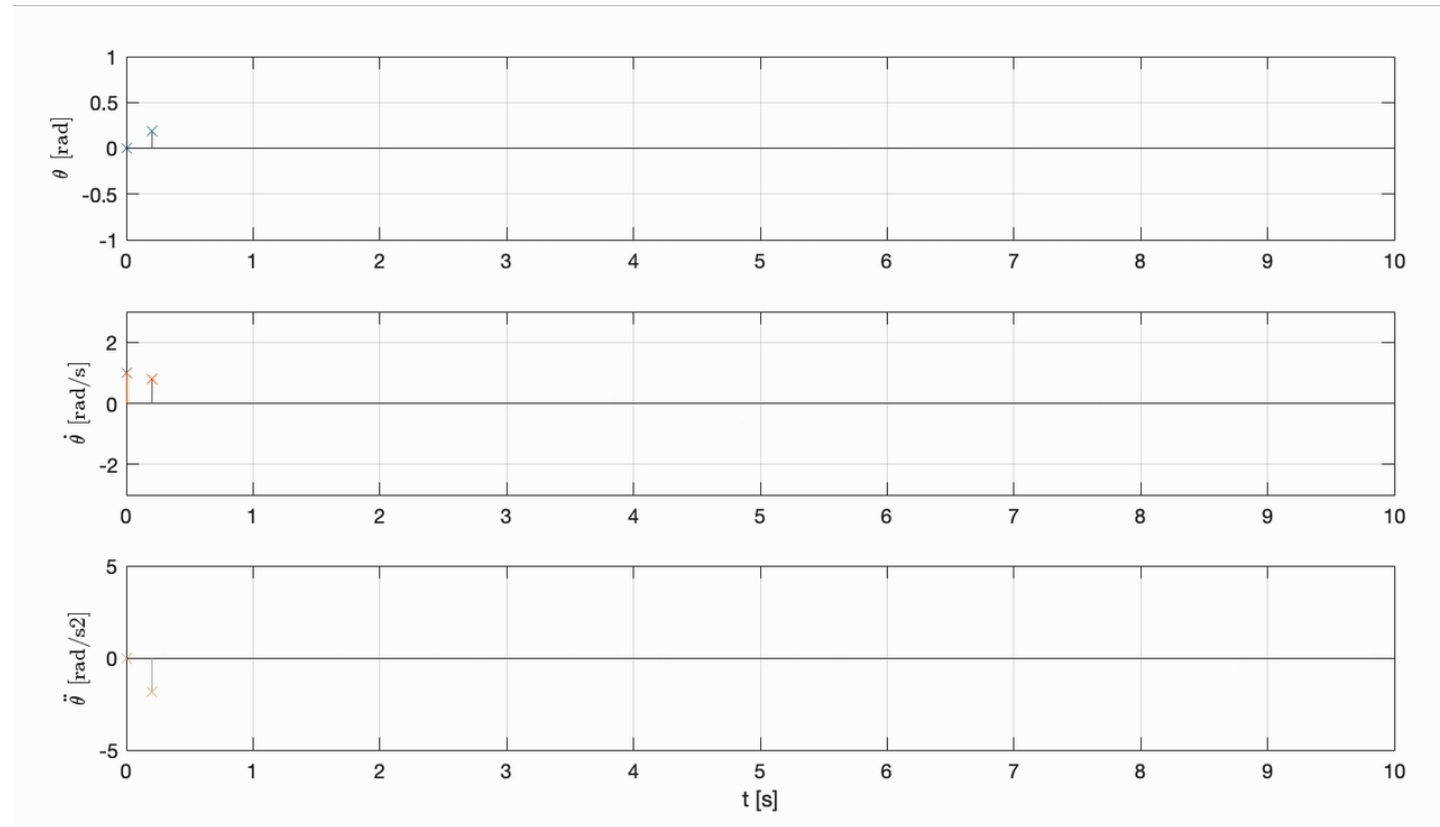
The analytical solution is extremely complex.

The alternative is to use **numerical integration**.

In numerical integration, we are not interested in the analytical form of the solution. We just want to know the values of the solution at specific time points.

Numerical Integration in a nutshell

- We start from the initial conditions
- We evaluate the differential equations to know the value of the derivatives
- Once we know the derivatives, we **approximate the function(s) locally** to find the next values. A formula we could use is:
$$x_{n+1} = x_n + \dot{x}_n \Delta t$$
- We go on with the procedure using the new values



Numerical Integration in a nutshell

With numerical integration, we only know the value of the function at specific time values.

There are different solvers to choose from. They differ from:

- The time step Δt : for some is **fixed**, for some other is **variable**
- The approximation of the derivative: some are more accurate (but more demanding), some less accurate (but faster)
- The method to compute the new values: some are simple (i.e. forward Euler), some more complex (but more accurate)
- And other peculiarities

Which integrator should I use? (1)

The choice of the integrator depends on the model we are simulating. Some golden rules:

1. If you are simulating continuous things (i.e., the world), variable step solvers are okay; however, you could incur into extremely small timesteps that slow your simulation down.
2. If you are simulating things that work on a clock (i.e., a sensor, a computer), you must set the sample time of these systems accordingly. This will influence the integrator step size.
3. The step size should be chosen wisely according to what you are simulating. «Faster» systems require lower step sizes, «slower» systems can accept larger step sizes

Which integrator should I use? (2)

4. The more steps you use, the slower your simulation will be (because the computer will need to calculate more and more values!)
5. Some industry standards require you to use a fixed-step solver in Simulink
6. If you plan to deploy your Simulink model (e.g., for a controller) on real-time hardware, then you must stick to fixed-step solvers because that's how your hardware will work.

To be on the safe side, in this course stick to fixed-step solvers, and remember to tune your step size carefully!

More info: <https://it.mathworks.com/help/simulink/ug/choose-fixed-step-solver.html>